Low-Rank Approaches for Continual Learning in LLMs

Harshal Kulkarni

NYU Tandon School of Engineering

Advisor: Prof. Anna Choromanska
Contributors: Kristi Topollai, Tolga Dimlioglu

May 2025

Motivation

- LLMs demonstrate state-of-the-art performance on NLP tasks but degrade when fine-tuned continually.
- Catastrophic forgetting arises from sequential learning without mechanisms to retain prior knowledge.
- Retraining is memory-intensive and computationally inefficient.
- Goal: Design parameter-efficient strategies to retain and integrate task knowledge incrementally.

Problem Statement

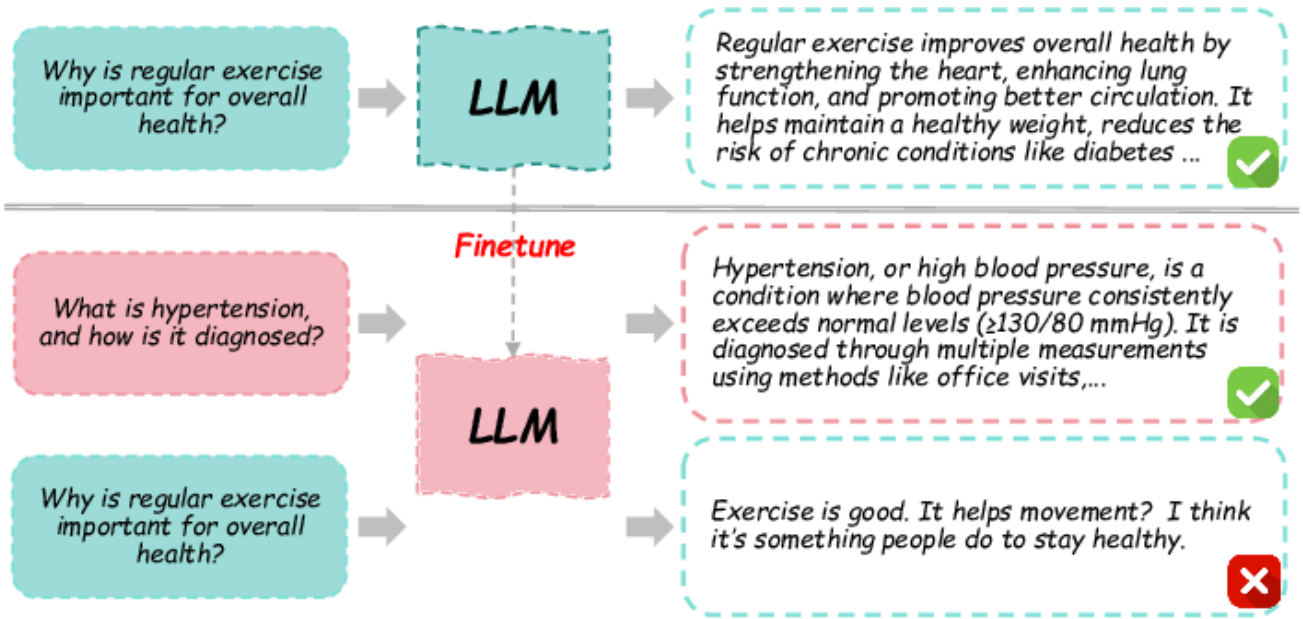
- Continual learning poses the challenge of learning tasks sequentially while preserving performance on prior tasks.
- Desiderata: No access to past data, efficient in memory and compute, generalize to unseen tasks.
- Key research questions:
  - Can we enforce weight-space separation to avoid interference?
  - Are task-specific weight updates additive or reusable?
  - What theoretical principles govern low-rank continual updates?

Research Contributions

- Developed a unified framework to apply and evaluate LoRA-based continual learning in LLMs.
- Analyzed O-LoRA: orthogonality-constrained subspaces for task isolation.
- Integrated Task Arithmetic for post-hoc multi-task composition and interpretability.
- Conducted extensive empirical validation across NLP benchmarks.

Continual Learning and Catastrophic Forgetting

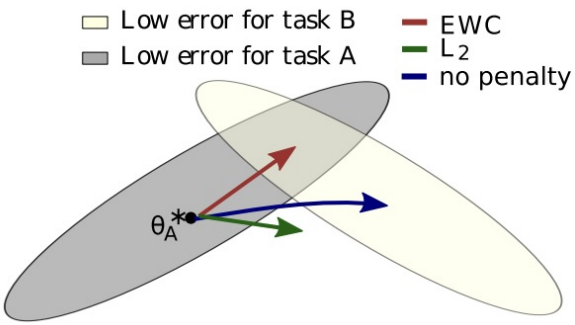
- **Continual Learning:** Learning a sequence of tasks $T_1, T_2, \ldots, T_n$ without forgetting previous ones.
- **Example:** A language model trained to classify news topics (e.g., sports), then updated for new topics (e.g., politics).
- **Challenge:** When training on T_2, performance on T_1 drops — this is **catastrophic forgetting**.



How Models Are Continually Trained

Continual learning strategies generally fall into three categories: **Replay-based**, **Regularization-based**, and **Architectural methods**.

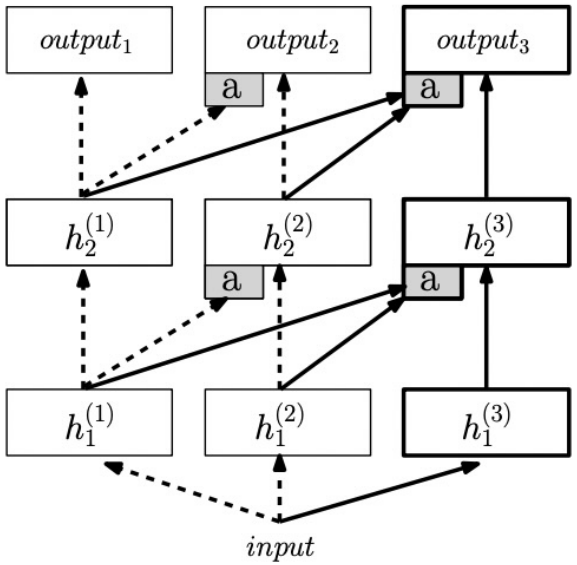
Regularization (EWC)



$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i(\theta_i - \theta^*_{A,i})^2$$

Kirkpatrick et al., 2017
Overcoming Catastrophic Forgetting in Neural Networks
`https://arxiv.org/abs/1612.00796`

Architecture



Rusu et al., 2016
Progressive Neural Networks
`https://arxiv.org/abs/1606.04671`

Continual Finetuning Scenarios

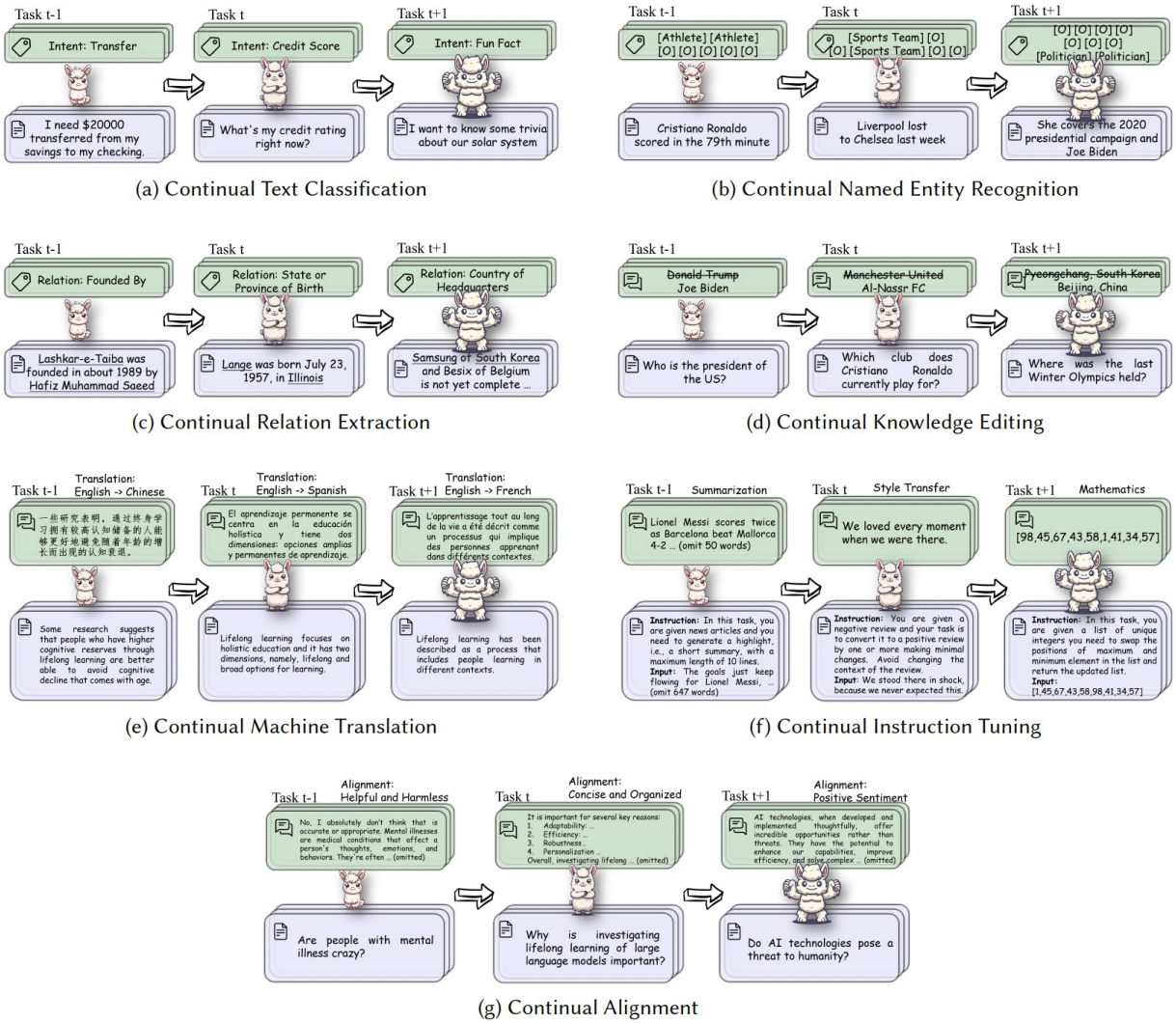


Figure: Examples of continual finetuning tasks across classification, NER, translation, instruction tuning, and alignment. Purple = input, Green = output.

Background: Continual Learning in LLMs

- Continual Learning enables adaptive task addition without retraining from scratch.
- Key paradigms:
  - **Task-Incremental**: Task identity known at inference.
  - **Class-Incremental**: Model generalizes across tasks without ID.
- Challenges:
  - **Forgetting**: New learning degrades previous performance.
  - **Interference**: Overlapping parameter space updates.
  - **Scalability**: Managing memory and compute budget.

CL Techniques for LLMs

- CL in LLMs includes diverse scenarios — task-, domain-, and class-incremental — differing in task identity, label space, and data format.
- We're no longer dealing with small networks. Modern LLMs have *billions of parameters*, making continual learning much more challenging due to increased scale.
- However that same large scale opens up new avenues.

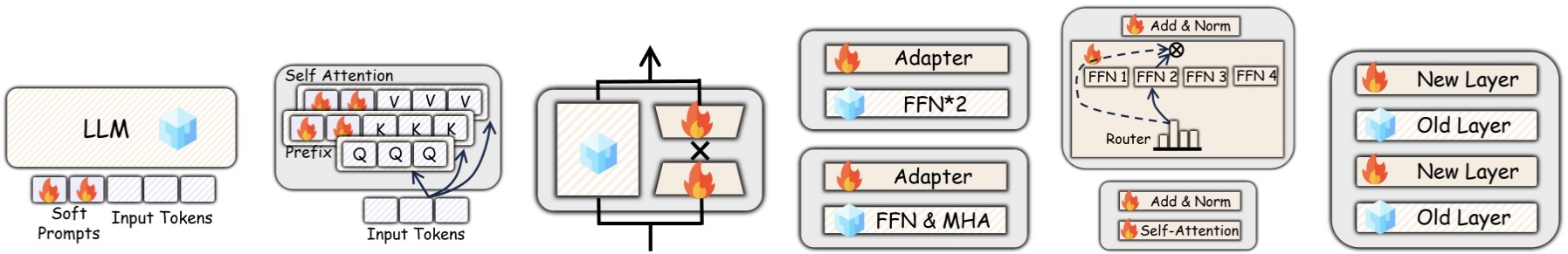


Figure: Six categories for continual learning for LLMs

Parameter-Efficient Fine-Tuning (PEFT)

- Fine-tune only subsets of parameters.
- **LoRA** fine-tunes LLMs by adding low-rank updates: $\Delta W = AB$, while keeping original weights fixed.
- $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ with small rank $r \ll d$.
- This injects only $2dr$ parameters per matrix, significantly reducing memory and compute cost.

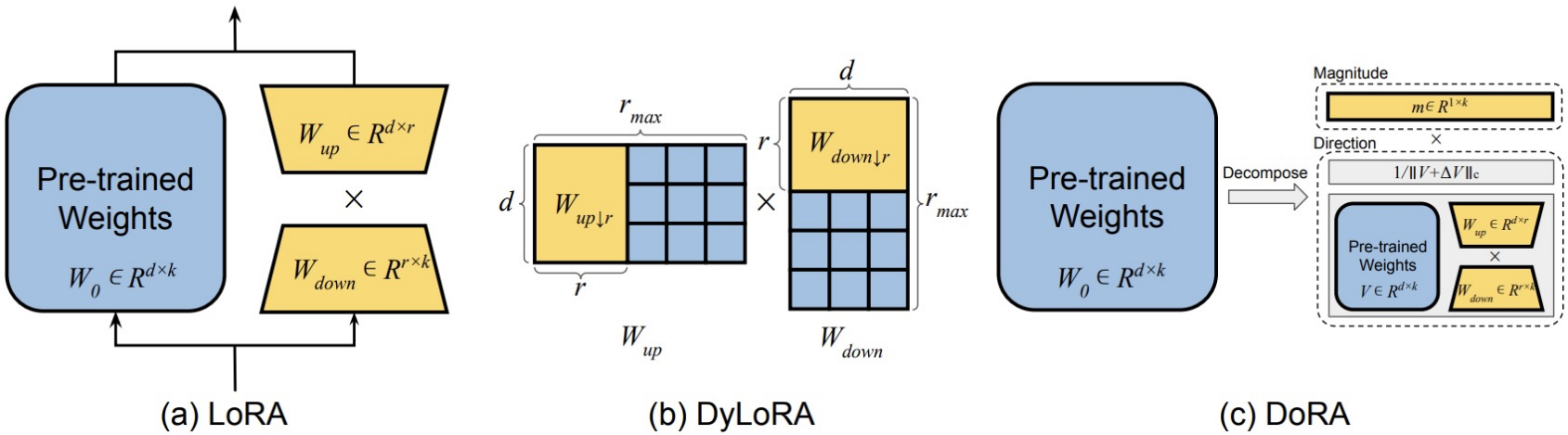


Figure: Common PEFT techniques including LoRA, DyLoRA, DoRA.

Reference: Sun, Z., Huo, Y., Liu, Y., Zhang, Q., Liu, Y. (2023). *Orthogonal Subspace Learning for Language Model Continual Learning*. arXiv preprint arXiv:2305.19081.

Would PEFT Make Sense in Continual Learning?

PEFT is well-suited for Continual Learning (CL) due to its ability to adapt models using minimal parameter changes. Introduces a small number of trainable parameters per task, avoiding full model retraining.

Example – Sequential LoRA:

- A simple PEFT-based baseline for CL that appends new LoRA adapters over base model layers.
- After each task, LoRA adapters are merged with base layer and continued with training for the next task.

Orthogonal Gradient Descent (OGD)

Orthogonal Gradient Descent (OGD) is a continual learning method that mitigates forgetting by constraining new gradients to be orthogonal to gradients from previous tasks.

Key Idea: Project the gradient of the new task onto the subspace orthogonal to all previous task gradients: $g_t^{\perp} = g_t - \sum_{i<t} \frac{g_t^{\top} g_i}{\|g_i\|^2} g_i$

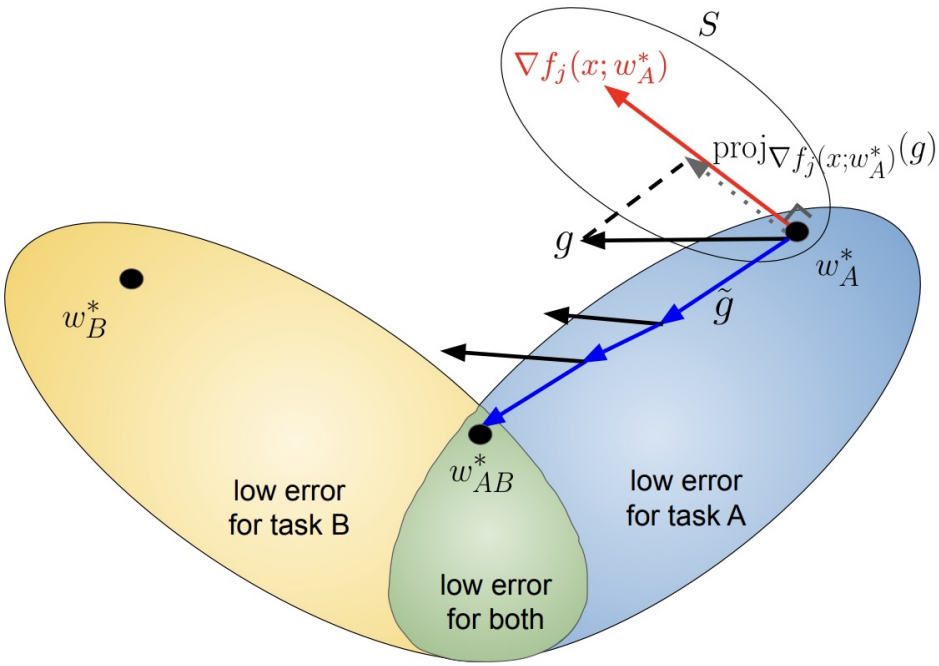


Figure: Visualization of OGD gradient projection onto the orthogonal subspace to past task gradients.

OGD and Its Connection to O-LoRA

Connection to O-LoRA: O-LoRA extends the OGD principle to the parameter-efficient setting of LoRA.

Rather than projecting gradients, they enforce orthogonality of low-rank update directions to isolate task-specific subspaces:

$$\mathcal{L}_{\perp} = \lambda \sum_{i<t} \left\| A_i^{\top} A_t \right\|^2$$

where A_t is the task-specific LoRA projection matrix at step t.

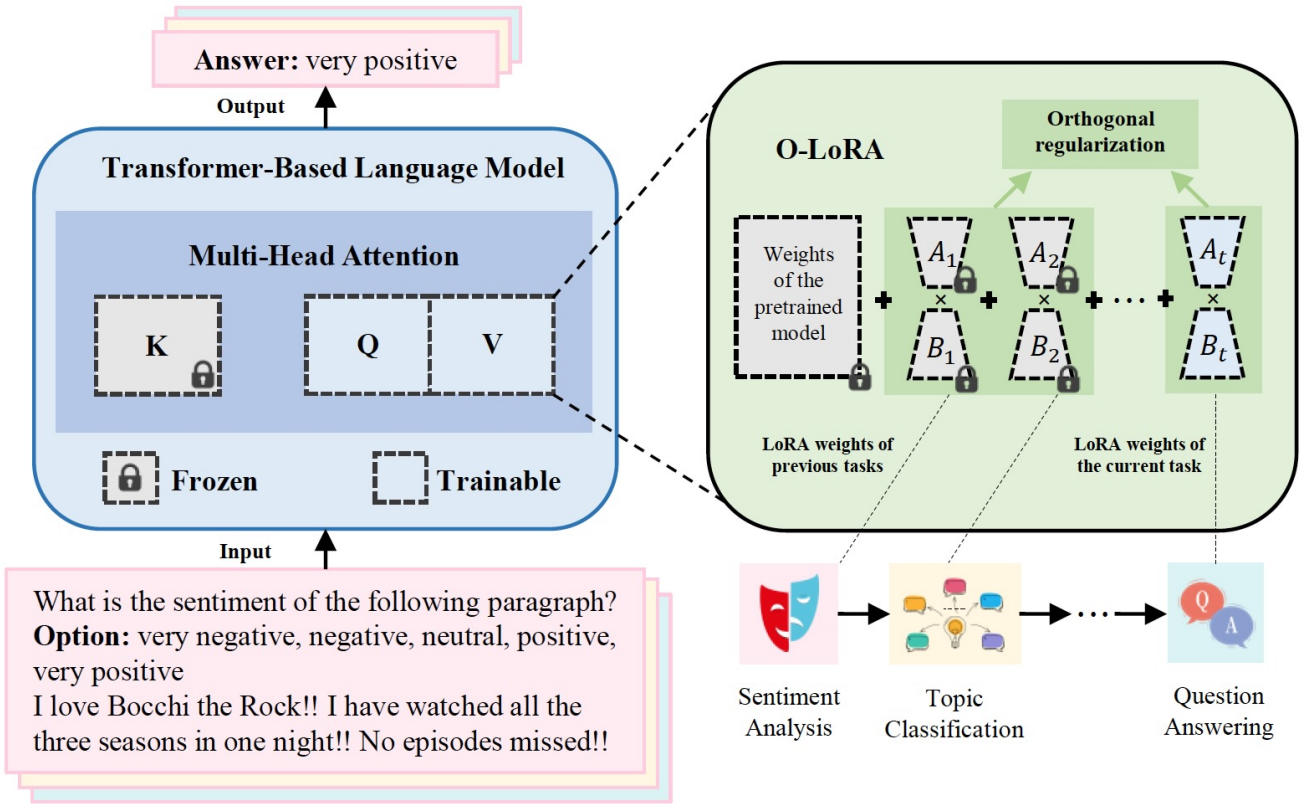
Reference: Farajtabar, M., Azizan, N., Mott, A., Li, A. (2020). Orthogonal Gradient Descent for Continual Learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*. arXiv:1910.07104

O-LoRA: Orthogonal Low-Rank Adaptation (1/2)

- **Core Idea:** O-LoRA learns task-specific low-rank updates in subspaces **orthogonal** to all previously learned tasks to reduce forgetting.
- For task t, LoRA update is defined as: $\Delta W^{(t)} = A^{(t)} B^{(t)}$.
- This defines a task-specific subspace: $S_t = \text{span}(A^{(t)})$.
- **Orthogonality constraint:** $S_t \perp S_i$ for all $i < t$, i.e., new task directions avoid interfering with earlier ones.
- Inspired by Orthogonal Gradient Descent — extends orthogonality from gradients to LoRA subspaces.

Reference: Sun et al. (2023). *Orthogonal Subspace Learning for Language Model Continual Learning*. arXiv:2305.19081.

O-LoRA: Architecture



O-LoRA enforces orthogonality among low-rank adaptation weights across tasks, enabling continual learning without forgetting.

O-LoRA: Orthogonal Low-Rank Adaptation (2/2)

- To enforce orthogonality, a regularization term is added to the loss:

$$\mathcal{L}_{\perp} = \lambda \left\| (A^{(i)})^{\top} A^{(t)} \right\|^2$$

- Full loss for task t becomes:

$$\mathcal{L}_t = \log p_{\theta}(y|x) + \lambda \sum_{i<t} \|A_i^{\top} A_t\|^2$$

- Encourages new LoRA directions $A^{(t)}$ to be orthogonal to all previous $A^{(i)}$.
- **Key Insight:** LoRA modules encode update *directions*, not just values. Preserving direction independence helps retain previous task knowledge.

Reference: Sun et al. (2023). *Orthogonal Subspace Learning for Language Model Continual Learning*. arXiv:2305.19081.

O-LoRA: Benchmark Performance and Generalization

- **Evaluation Setup:**
  - Base Model: T5-Large (3B parameters)
  - Benchmarks:
    - 5-Task Sequence: AG News, Amazon, Yelp, DBpedia, Yahoo.
    - 15-Task Sequence: Includes GLUE, SuperGLUE tasks.
- **Performance Results:**
  - **5-Task Avg. Accuracy:** O-LoRA ∼75.8% vs LFPT5 ∼72.7%; approaching joint training upper bound (80%).
  - **15-Task Avg. Accuracy:** O-LoRA ∼69.6%, outperforming:
    - EWC (∼45%), L2P (∼56%), Progressive Prompt (∼77.9%, but needs task ID).
  - **O-LoRA requires only a single model at inference — no task ID needed.**
- **Generalization to Unseen Tasks:**
  - Model: LLaMA-7B fine-tuned on Alpaca, then trained on 5-task sequence.
  - Evaluated on **MMLU (zero-shot)**:
    - Naive LoRA: 23.3%, near random.
    - O-LoRA: Mid-30s%, close to Alpaca base (37.5%).
    - Example: Medical domain – O-LoRA: 36.1% vs base 40.9%, naive

Implementation Details

- **Model:** OPT-1.3B Decoder-only model trained for **causal language modeling** — predicts next-word tokens.
- **Tokenizer:** 'GPT2Tokenizer'
- Encodes/decodes text into token sequences compatible with OPT models.
- **Framework:** DeepSpeed + accelerate for multi-gpu training on NYU HPC.
- **Datasets and Task Types:**
  - `DBpedia` – Topic Classification (14 classes)
  - `Amazon` – Sentiment Analysis (5-star product reviews)
  - `Yahoo` – Question Classification (10 categories)
  - `AGNews` – News Topic Classification (4 classes)
- **Validation Metrics:**
  - **Accuracy:** Exact match between generated and ground truth responses.
  - **ROUGE-L:** Measures overlap between predicted and reference text spans.

O-LoRA: Empirical Results

- Evaluation on 4 NLP tasks with OPT-1.3B under sequential task setting.
- **JOINT model** serves as upper bound with multitask training.
- **Seq LoRA:** same LoRA layers reused, suffers from severe forgetting.
- **Inc LoRA:** merges previous adapters, modest improvement.
- **O-LoRA ($\lambda = 0.5$):** yields major gains via orthogonality constraints.

Method	DBpedia	Amazon	Yahoo	AGNews	Avg. Acc.
JOINT (Upper Bound)	98.67	73.37	74.43	89.57	84.01
Seq LoRA	29.87	58.12	52.09	90.95	57.76
Inc LoRA	31.03	58.58	56.12	89.71	58.86
O-LoRA ($\lambda = 0.1$)	45.50	47.66	55.57	90.25	59.75
O-LoRA ($\lambda = 0.5$)	68.02	38.60	56.93	90.23	63.44

Modified O-LoRA: Orthogonality in Effective Space

Why modify O-LoRA? While O-LoRA enforces orthogonality in the LoRA projection space ($A_i^\top A_j = 0$), it does not guarantee orthogonality in the actual update space $\Delta W = AB$. Thus, even if $A_i \perp A_j$, the product $A_i B_i$ may still interfere with $A_j B_j$, especially if B matrices are unconstrained.

Key Insight: To truly avoid task interference, we should ensure that the full low-rank updates for different tasks are orthogonal: $(A_i B_i)^\top (A_j B_j) = 0$.

Modified O-LoRA (MLoRA):

- Replaces constraint $A_i^\top A_j = 0$ with $(A_i B_i)^\top (A_j B_j) = 0$
- Operates directly in the effective update space, aligning better with downstream model behavior. Can lead to improved generalization and task retention empirically

Results: Compared to Sequential LoRA and O-LoRA ($\lambda = 0.5$)
MLoRA shows substantial gains over naive sequential training.

Method	**DBpedia**	**Amazon**	**Yahoo**	**AGNews**	**Avg. Acc.**
Seq LoRA	29.87	58.12	52.09	90.95	57.76
O-LoRA ($\lambda = 0.5$)	68.02	38.60	56.93	90.23	63.44
Modified O-LoRA (Ours)	72.86	48.11	58.68	89.84	67.37

Is O-LoRA necessary?

- Cosine similarity between task vectors $A_i B_i$ across tasks is close to zero:

$$\cos(\theta_{i,j}) = \frac{\langle A_i B_i, A_j B_j \rangle}{\|A_i B_i\| \|A_j B_j\|} \approx 0$$

- Indicates emergent orthogonality even without explicit regularization.

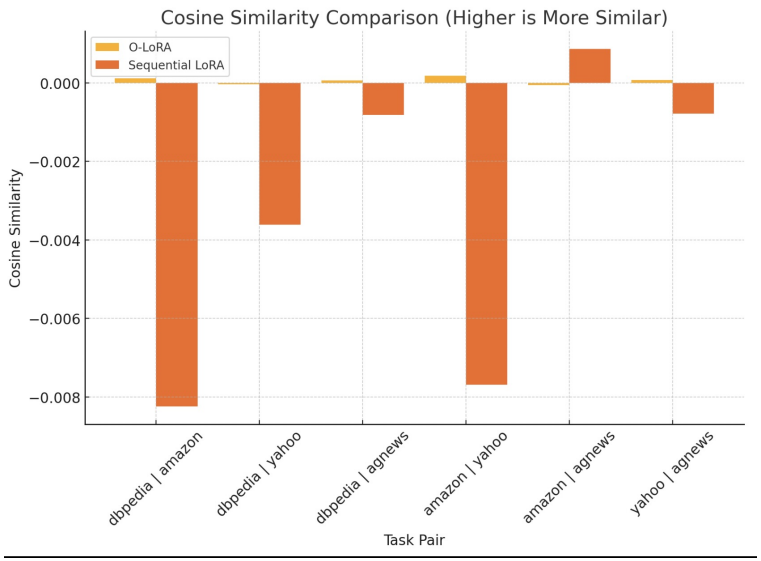


Figure: Cosine similarity distribution among task vectors. Most are centered around zero.

Random Vectors are Almost Orthogonal

Theorem. Let $X, Y \in \mathbb{R}^d$ be *independent, isotropic* random vectors with sub-Gaussian coordinates (variance 1). Then for universal $c, C > 0$,

$$\Pr\big(|\cos\theta| \geq \varepsilon\big) \;\leq\; C\exp\big(-c\,\varepsilon^2 d\big), \qquad \theta = \angle(X, Y).$$

Main tool $\rightarrow$ **Bernstein's inequality** for sums of independent sub-exponential variables (used on $\langle X, Y\rangle$ and $\|X\|^2, \|Y\|^2$).

Interpretation.

- The probability of the angle between two such random vectors being significantly far from 90° (i.e., $|\cos\theta|$ being noticeably large) drops off exponentially fast in d.
- Equivalently, the angle concentrates around $\frac{\pi}{2}$ at an $\exp(-\Omega(d))$ rate.
- Any fixed set of $k \leq e^{\alpha d}$ independent isotropic vectors is pairwise ε-orthogonal w.h.p.

What is Task Arithmetic

Task merging works surprisingly well: Adding multiple task-specific weight updates ("task vectors") to a base model often yields a single multi-task model that performs well on all tasks — sometimes even outperforming models fine-tuned on each task individually. This suggests joint training isn't always necessary.

- An emergent phenomenon in Large Models.
- Allows for effective multi-task learning.
- What about continual learning?

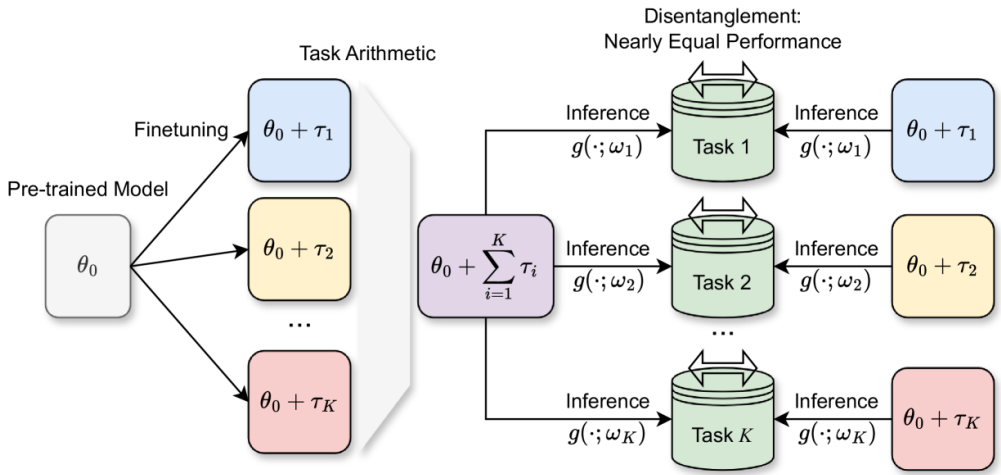


Figure: Merging models finetuned on different tasks retains the performance on the individual tasks

Motivation for Task Arithmetic

- **Minimal task interference:** The merged model typically retains 96–97% of the original task performance, showing that task vectors do not conflict heavily.
- **Positive cross-task transfer:** Merging an external task vector can enhance single-task performance. For instance, on GLUE text classification tasks, a T5 model fine-tuned on one task gained an additional $\sim$0.5% accuracy on average by incorporating a second task vector from a public checkpoint — beyond what fine-tuning alone achieved.
- **Task vector:** $\tau_t = \theta_t^* - \theta_0$ — the weight delta between fine-tuned and pre-trained models for task t.
- **Task arithmetic:** Combining task vectors (e.g., $\theta_0 + \tau_A + \tau_B$) to yield a model that performs well on both tasks A and B.

Task Arithmetic – Foundations and Findings

- Initially attributed to neural networks operating in a linear regime (Wortsman et al., via NTK theory).
- Ilharco et al. (2023) **disprove** this hypothesis: fine-tuned CLIP models operate outside the linear NTK regime.
- Key finding: **Weight disentanglement** — task vectors affect disjoint regions in input space, enabling effective combination.
- NTK linearization **amplifies** but is not essential for task arithmetic to work.

References:
Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., Farhadi, A. (2023). *Editing Models with Task Arithmetic*.

Weight Disentanglement Enables Task Arithmetic

- Formally, a model is weight disentangled if:

$$f\left(x;\theta_0+\sum_t \alpha_t\tau_t\right)=\sum_t g_t(x;\alpha_t\tau_t)+g_0(x)$$

where each g_t vanishes outside task-specific domain D_t.

- **Emergent property:** Absent at random initialization — appears only after pre-training.
- This disentanglement is enhanced via NTK-based linearization but exists even in non-linear models.

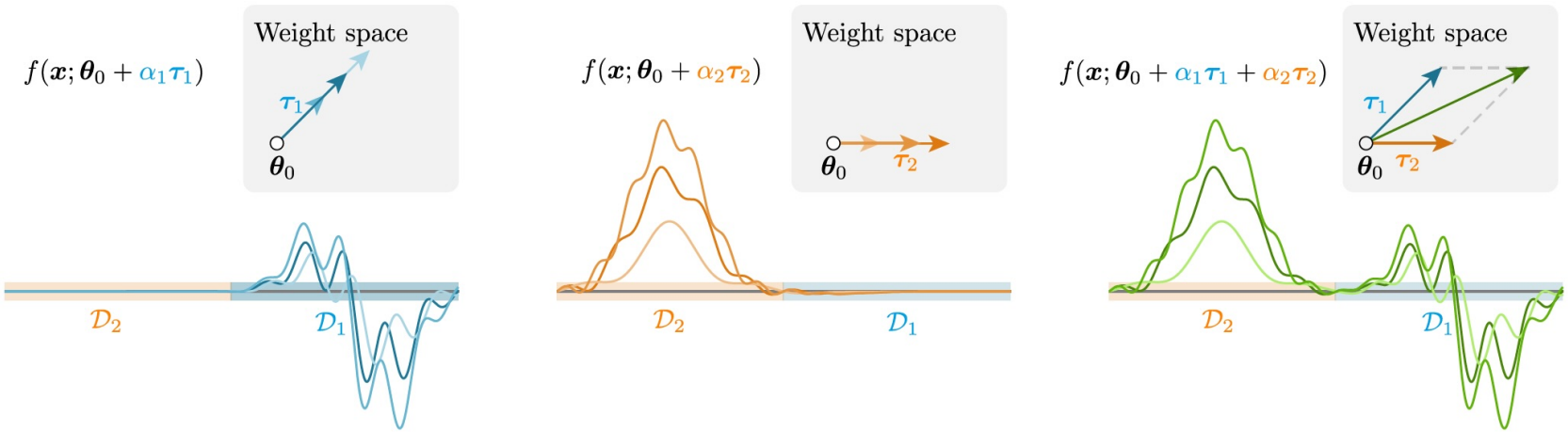


Figure: Disjoint task regions influenced by separate weight directions.

Orthogonality of Task Vectors

- **Task interference measure:** For each task i, define the *merging gap* G_i as the change in task i's loss when other task updates are applied:

$$G_i = L_i\left(\theta + \sum_j \lambda_j \tau_j\right) - L_i(\theta + \lambda_i \tau_i)$$

  where τ_j is the task vector for task j and λ_j is a scaling factor.

- **First-order approximation:** Using Taylor expansion, the paper shows:

$$G_i \approx k_i \sum_{j \neq i} \lambda_j \langle \tau_i, \tau_j \rangle$$

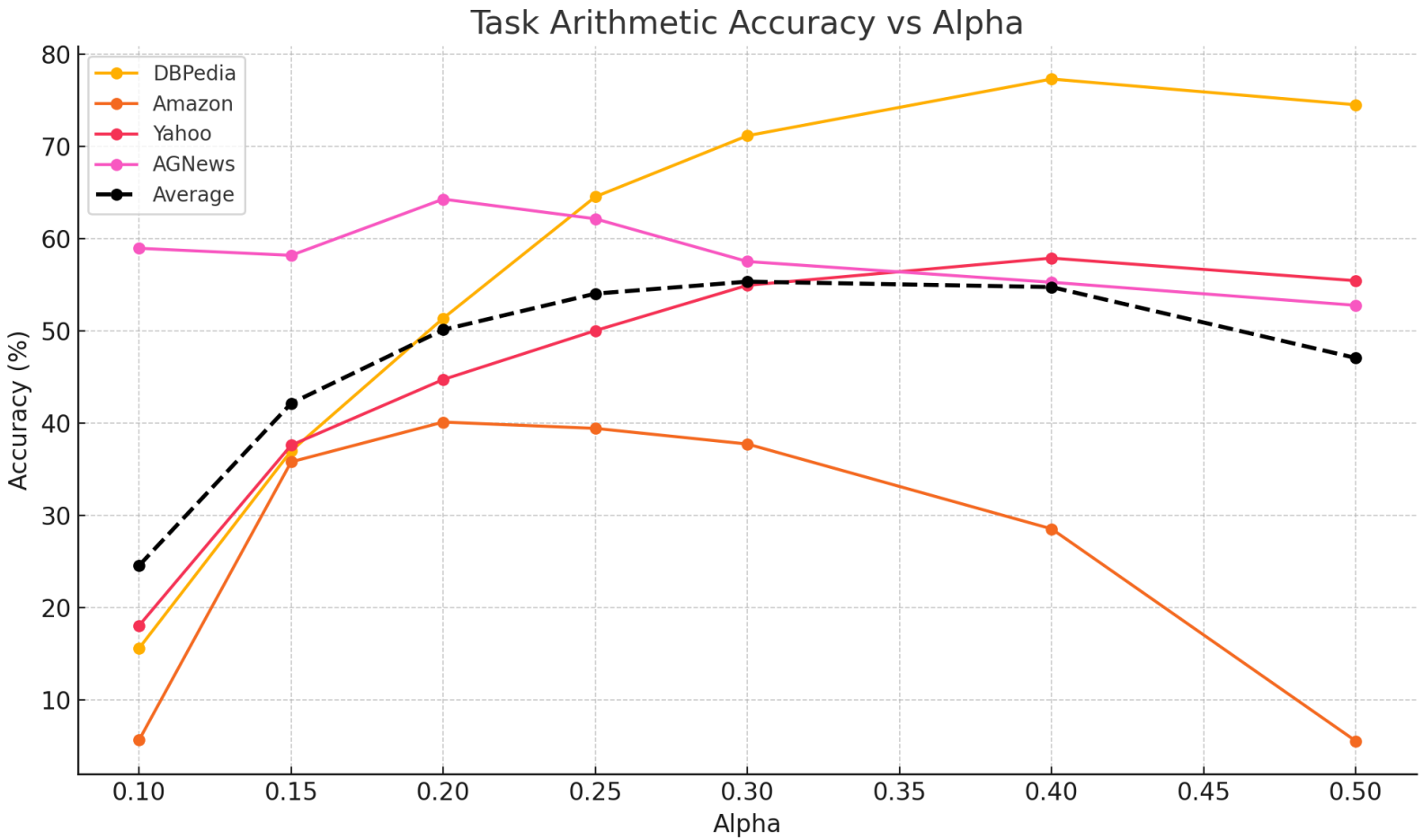
  where $k_i < 0$ is a constant depending on task i's curvature. Interference is thus proportional to the cosine similarity between task vectors.

- **Key Insight:** If $\langle \tau_i, \tau_j \rangle = 0$ for all $i \neq j$ (i.e., task vectors are orthogonal), then $G_i \approx 0$ — implying no cross-task interference.

- **Conclusion:** Orthogonality among task vectors guarantees the *Task Consistency* property and motivates enforcing such structure in continual learning (e.g., via O-LoRA).

Task Arithmetic Results on Benchmark Datasets

- α controls the interpolation weight applied to all previous task LoRA updates.
- The update rule is: $h' = (W_0 + \alpha(A_1B_1 + A_2B_2 + A_3B_3 + A_4B_4))\,x$
- Accuracy varies non-linearly with α across datasets.



Orthogonality Regularization for Task Arithmetic

Motivation: Task arithmetic assumes that task-specific updates (task vectors) occupy independent subspaces. However, in practice, naive LoRA updates may interfere when combined.

Our Approach:

- During single-task fine-tuning of the base model, we impose **orthogonality constraints** on the low-rank update direction $\tau_t = A_t B_t$ with respect to all previous task vectors $\{\tau_i\}_{i<t}$:

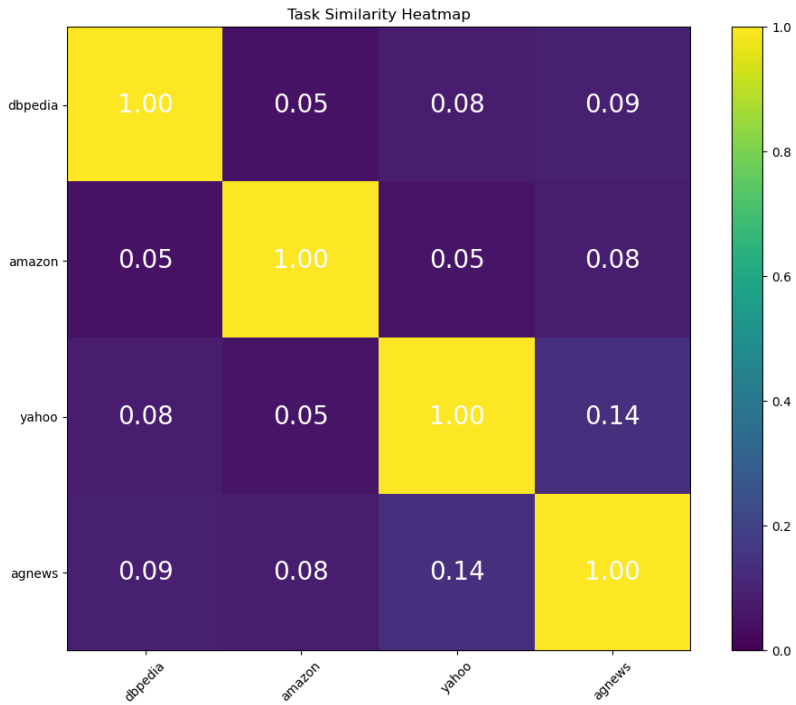
$$\mathcal{L}_{\perp} = \lambda \sum_{i<t} \left\| \frac{\tau_i^{\top} \tau_t}{\|\tau_i\| \|\tau_t\|} \right\|^2$$

- This regularization is added to the training objective to encourage each task vector τ_t to be orthogonal to all prior task vectors, minimizing overlap and interference.
- After training all tasks, we perform **task arithmetic** by merging the task vectors:

$$\theta^* = \theta_0 + \alpha \sum_{t=1}^{T} \tau_t$$

Task Arithmetic: Orthogonal Regularization Effect ($\alpha = 0.25$)

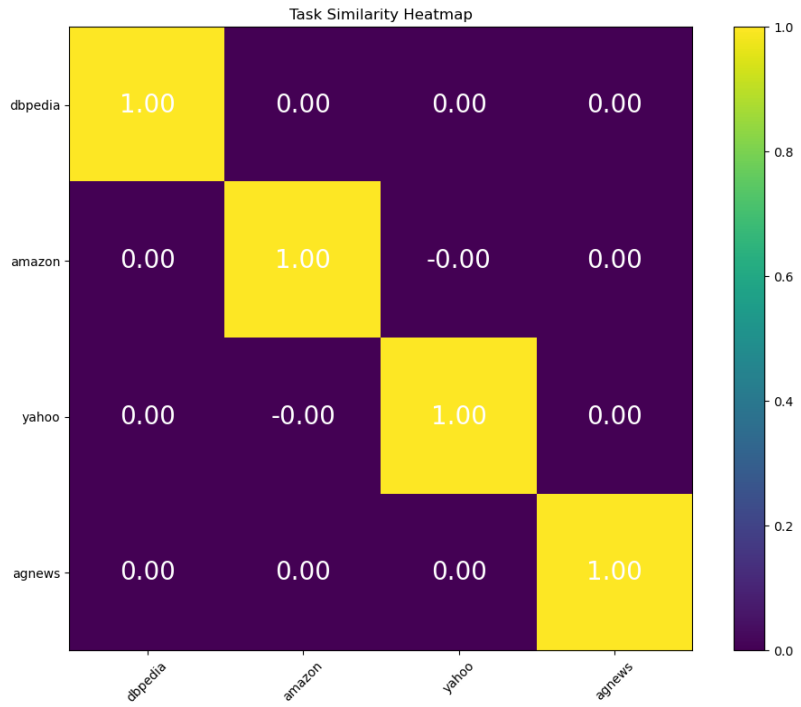
Without Orthogonal Regularization



Orthogonality heatmap for original fine-tuned task vectors.

Avg. Accuracy after merging: 54.03

With Orthogonal Regularization



Orthogonality heatmap for **orthogonally** fine-tuned task vectors.

Avg. Accuracy after merging: 57.35

Task Arithmetic: Empirical Analysis

- Task Arithmetic is extended to LoRA using $A_i B_i$ as task vectors.
- Merged using weighted sum: $\theta_0 + \sum \alpha A_t B_t$.
- We tested various $\alpha \in [0.1, 0.5]$ with and without orthogonal regularization.
- Orthogonal variants yield better stability and accuracy.

Method	DBpedia	Amazon	Yahoo	AGNews	Avg. Acc.
Single Task	98.66	73.06	72.47	88.09	83.07
Orth. Reg. STL	98.66	59.07	73.21	88.33	79.82
TA ($\alpha = 0.25$)	64.53	39.42	50.02	62.14	54.03
TA + Orth. Reg. ($\alpha = 0.25$)	70.29	38.68	56.10	64.34	**57.35**
TA ($\alpha = 0.3$)	71.15	37.72	54.94	57.51	55.33
TA ($\alpha = 0.4$)	77.30	28.53	57.88	55.26	54.74
TA ($\alpha = 0.5$)	74.51	5.53	55.43	52.76	47.06

Limitations

- Task Vectors AB size is same as attention matrix size. Due to the presence of in regularization term, as the number of tasks increases, training becomes computationally expensive.
- Deep transformer layers still show interference.
- Scaling beyond 20+ tasks not fully efficient.

Conclusion

- **O-LoRA** introduces orthogonal subspace training. Despite explicit constraints, we found that task vectors are already fairly orthogonal.
- Our orthogonality-based regularization confirms and enhances this emergent structure.
- In task arithmetic, merging separate and orthogonal task vectors performs worse than using continually trained task vectors — contrasting with the findings of **Ilharco et al. (2023)**, who report performance *improvement* when combining fully fine-tuned models via task vectors.

Reference: Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., Farhadi, A. (2023). *Editing Models with Task Arithmetic*. arXiv:2212.04089. `https://arxiv.org/abs/2212.04089`

Continual Task Arithmetic via SD-LoRA

- SD-LoRA reuses low-rank matrices from previous tasks via weighted interpolation.
- $\Delta W = \alpha_1 A_1 B_1 + \alpha_2 A_2 B_2 + \cdots + \alpha_j A_j B_j$
- Normalized task directions enable memory-efficient continual learning.

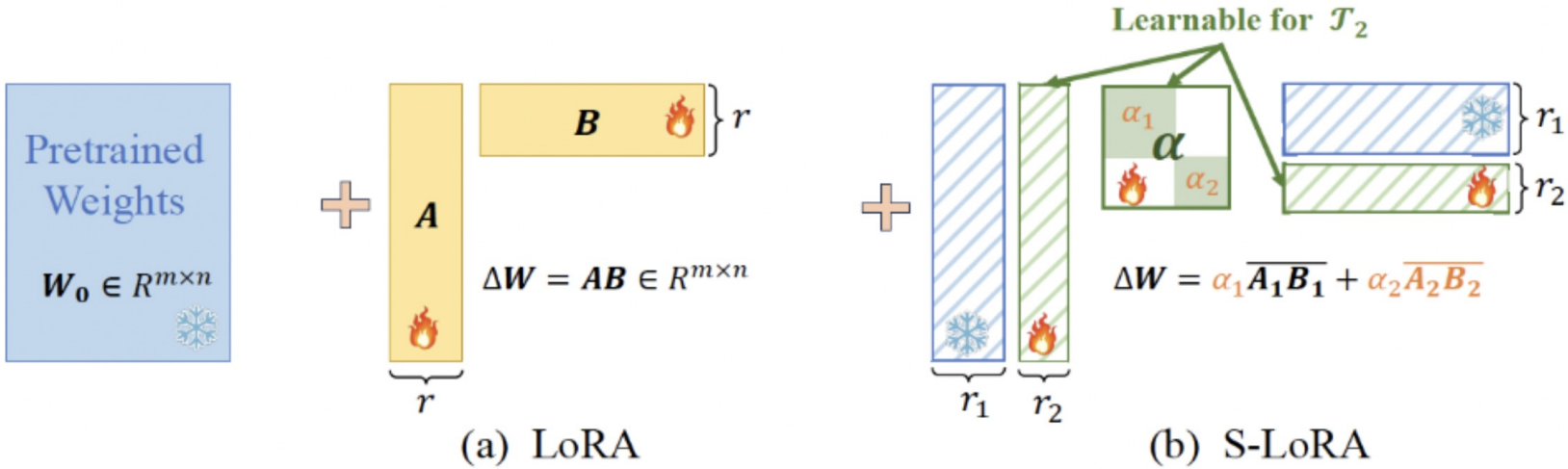


Figure: S-LoRA reparameterizes weight updates by reusing orthogonalized LoRA directions from previous tasks. Each $A_i B_i$ is normalized and scaled with learnable α_i.

Future Work

- Try SD-LoRA(applied to vision) on current NLP dataset and model. textcolorredYou can raise some questions and interesting points about the interesting effectivity of SD-LoRA

Thank You!

Questions?

harshal.kulkarni@nyu.edu